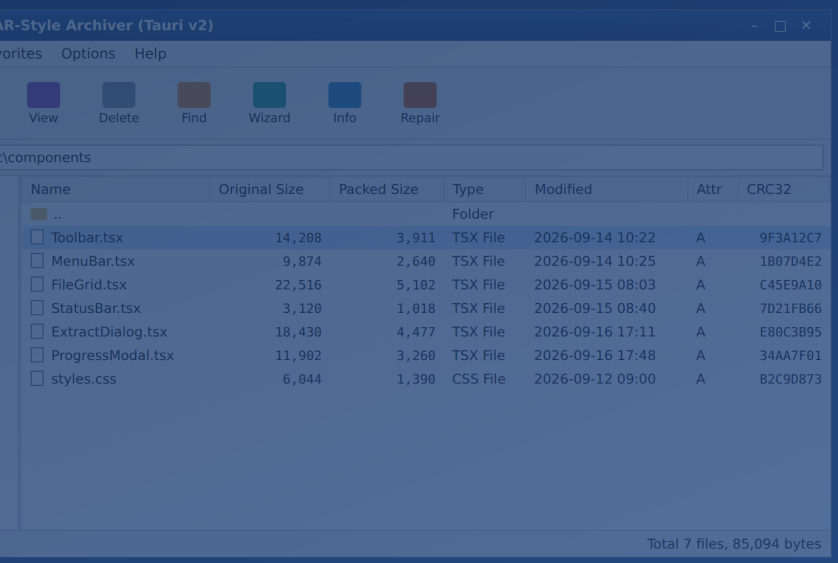
التوصيف المعماري والتنفيذي لاستنساخ منظومة WinRAR عبر هندسة البرمجة الموجهة

تفكيك المعمارية الهندسية، وتفصيل واجهة المستخدم، واستراتيجية التنفيذ المعاصرة باستخدام Tauri v2 ولغة Rust



Vibe Coding | Tailwind CSS | React 19 | Rust | Tauri v2

فهرس المحتويات

1. التحليل الوظيفي والمعماري لمحرك الأرشفة
2. الهندسة التفكيكية لواجهة وتجربة المستخدم
3. الحوارات التفاعلية المتقدمة ونماذج التشغيل
4. المعمارية التقنية للمستنسخ الحديث
5. إطار عمل واستراتيجية البرمجة الموجهة (Vibe Coding)
6. الهندسة الداخلية لملفات الأرشيف ذاتي الاستخراج (SFX)
7. الاستنتاجات الهندسية والتوصيات التنفيذية

> **ملاحظة حول الصور:** الصور الواردة في هذا التقرير هي نماذج توضيحية (Mockups) أُعدّت خصيصاً لتمثيل شكل الواجهة المقترحة للمستنسخ وفق المواصفات الواردة هنا، وليست لقطات شاشة من برنامج WinRAR الأصلي.

يتناول هذا التقرير تفكيك المعمارية الهندسية الشاملة لبرمجية WinRAR، وتفصيل كافة متطلبات الواجهة الرسومية وتجربة المستخدم، واستعراض استراتيجية تقنية معاصرة تعتمد على Tauri v2 ولغة Rust، مع تقديم إطار عمل هندسي لتنفيذ الاستنساخ بأسلوب البرمجة الموجهة (Vibe Coding) عبر بيئات التطوير الذكية الموجهة بالذكاء الاصطناعي.

1 التحليل الوظيفي والمعماري لمحرك الأرشفة

تمثل برمجية WinRAR إحدى الركائز التاريخية في بيئات الحوسبة المكتبية لأنظمة ويندوز منذ إطلاقها الأول عام 1995 على يد المهندس يوجين روشال (Eugene Roshal). ولم يكن استمرارها لعقود مبنياً على مجرد وظيفة ضغط الملفات، بل على تكامل عميق مع معمارية نظام التشغيل، وخوارزميات أرشفة مصممة ذات كفاءة غير متماثلة، ونظم تصحيح أخطاء رياضية متطورة.

1.1 خوارزميات الضغط وآليات إدارة الذاكرة

ترتكز الكفاءة الجوهرية للتنسيق الاحتكاري RAR على توليفة هجينة مشتقة من خوارزميات Lempel-Ziv (LZSS) وترميز Huffman، مقترنة بخوارزميات ترشيح متخصصة للبيانات القابلة للتنفيذ (IA-32 Executables) والبيانات الثنائية. شهدت المنظومة قفزات بنيوية متتالية؛ حيث اعتمد معيار RAR4 على قواميس انزلاقية لا تتجاوز 4 ميجابايت، في حين أحدث إطلاق معيار RAR5 نقلة نوعية برفع الحجم الافتراضي للقاموس إلى 32 ميجابايت مع إمكانية التوسع حتى 1 جيجابايت في بيئات 64-بت.

في الإصدار السابع (WinRAR 7.00)، اتسعت حدود القاموس الانزلاقي لتصل إلى 64 جيجابايت. يتيح هذا الاتساع التقاط الأنماط المتكررة المتباعدة في مجموعات البيانات الضخمة، كملفات الصور الافتراضية وقواعد البيانات، وهو ما تعززه خوارزميات البحث بعيد المدى (Long Range Search) والبحث الاستقصائي الشامل (Exhaustive Search) التي أضيفت لتقليص نسب الضغط واكتشاف التماثل عبر كتل البيانات المتباعدة بملايين البايتات. في المقابل، يظل تنسيق ZIP المنشأ بواسطة البرنامج محكوماً بحدود القاموس التقليدي البالغ 32 كيلوبايت.

تعتمد البرمجية مفهوم "الأرشفة المصمتة" (Solid Archiving)، وهي تقنية معمارية تُعامل مجموعة الملفات المدخلة كتيار بياني متصل ومستمر بدلاً من معالجة كل ملف في كتلة بيانات مستقلة. ينتج عن هذا الأسلوب تحسين استثنائي في معدلات الضغط عند التعامل مع مجموعات هائلة من الملفات المتجانسة أو الصغيرة، غير أنه يفرض ضريبة أداء حوسبية تتمثل في بطء الوصول العشوائي؛ إذ يقتضي استخراج ملف فردي يقع في نهاية الأرشيف فك تشفير وتدفق كافة البيانات التي تسبقه ضمن الكتلة المصمتة ذاتها.

يدعم محرك الاستخراج طيفاً واسعاً من الصيغ يمتد ليشمل 7z، TAR، GZ، BZ2، XZ، Zstandard، LZH، ARJ، ISO، CAB، وZIPX، مع اقتصار محرك الإنشاء على صيغتي RAR وZIP، وحصر إنشاء الأرشيفات الحديثة في بيئة RAR5 وRAR7 بعد إسقاط دعم إنشاء صيغة RAR 4.x القديمة لتعزيز معايير الأمان والتوافق.

1.2 منظومة النزاهة والتشفير والتعافي من التلف

تتفوق معمارية RAR على نظيراتها من التنسيقات العامة بامتلاكها بنية تحتية مخصصة للصمود أمام التلف الفيزيائي وقنوات النقل غير المستقرة. يُدمج النظام ما يُعرف بـ "سجلات التعافي" (Recovery Records - RR) المستندة رياضياً إلى أكواد تصحيح الخطأ "ريد-سولومون" (Reed-Solomon). تتيح هذه السجلات للمستخدم تحديد نسبة استرجاع مئوية تُخزن ككتل تماثلية، مما يمكّن البرمجية من استعادة المحتوى وإصلاحه ذاتياً عند تعرض الملف لتلف في القطاعات التخزينية أو انقطاع الحزم الشبكية. كما تدعم المنظومة توليد "مجلدات التعافي" (`.REV` - Recovery Volumes) التي تعمل كبدائل تماثلية مستقلة قادرة على إعادة بناء أي جزء مفقود بالكامل عند تجزئة الأرشيف إلى مجلدات متعددة.

من الناحية الأمنية، تعتمد إصدارات RAR5 وRAR7 تشفير AES المتوافق مع معيار 256 بت بنمط ربط الكتل المشفرة (CBC Mode) مع تعقيد اشتقاق المفاتيح باستخدام خوارزميات إبطاء الهجمات القسرية، متفوقة على معيار AES-128 المطبق في RAR4. تتيح المنظومة خيار تشفير جداول المحتويات وأسماء الملفات بالكامل عبر المعامل `-hp`، وهو ما يمنع فحص البنية الشجرية للأرشيف دون التحقق المسبق من صحة كلمة المرور.

أما على صعيد فحص النزاهة الهيكلية، فقد دمجت المعمارية تقنية التجزئة المشفرة BLAKE2sp بطول 256 بت كبديل متقدم لرموز التدقيق الدوري التقليدية (CRC32)، مما يقضي على احتمالات تصادم التجزئة ويوفر آلية موثوقة للتحقق من خلو الملفات من التلاعب الخبيث. وتكتمل المنظومة الأمنية بميزة نقل "علامة الويب" (Mark of the Web - MOTW)، التي تعمل على تمرير مناطق أمان نظام التشغيل من الملف المضغوط إلى كافة الملفات المنبثقة عنه لتقييد تنفيذ البرمجيات غير الموثوقة.

1.3 ميزات نظام الملفات والبيانات الوصفية

تتكامل البرمجية بعمق مع خصائص أنظمة الملفات المتقدمة، وبخاصة نظام NTFS. تدعم المعمارية تخزين مسارات الملفات بطول يصل إلى 65,535 حرفاً بترميز UTF-8 القياسي لتفادي أخطاء تجاوز سعة المسار. كما تحافظ على الطوابع الزمنية عالية الدقة (بدقة 100 نانوثانية) بتنسيق التوقيت العالمي المنسق (UTC)، متفادية اضطرابات التوقيت المحلي عند تبادل الملفات عبر المناطق الجغرافية.

يدعم المحرك استبقاء الروابط الصلبة والرمزية (Hard & Symbolic Links)، ونقاط إعادة التحليل (Reparse Points)، وقوائم التحكم في الوصول الأمني (NTFS ACLs). وتتضمن المعمارية محركاً داخلياً لإلغاء التكرار (File Deduplication)، يعمل على تخزين النسخ المتطابقة من الملفات كإشارات

مرجعية تشير إلى النسخة الأولى داخل الأرشيف، مما يقلص الحجم الإجمالي دون إهدار الموارد الحوسبية.

الخاصية الفنية	معيار RAR4 القديم	معيار RAR5 / RAR7 الحديث	معيار ZIP القياسي
الحجم الأقصى للقاموس	4 ميجابايت	1 جيجابايت (RAR5) / 64 جيجابايت (RAR7)	32 كيلوبايت
خوارزمية التشفير الأساسية	AES-128 (CBC)	AES-256 (CBC) مع مفاتيح مشتقة محسنة	ZipCrypto ضعيف / WinZip AES اختياري
آلية التحقق من التكامل	CRC32 بطول 32 بت	BLAKE2sp بطول 256 بت أو CRC32	CRC32 بطول 32 بت
تصحيح الخطأ والتعافي	Reed-Solomon أولي	Reed-Solomon متقدم متعدد المقاومة	غير مدعوم بنيوياً
تشفير أسماء الملفات	مدعوم	مدعوم مع آليات تحقق سريعة من كلمة المرور	غير مدعوم بالمعيار الافتراضي
دقة الطوابع الزمنية	توقيت محلي قياسي	توقيت عالمي UTC بدقة 100 نانوثانية	توقيت MS-DOS محلي محدود الدقة
الحد الأقصى لطول المسار	2,047 حرفاً	65,535 حرفاً بصيغة UTF-8	260 حرفاً (بدون ملحقات النظام الممتدة)

2 الهندسة التفكيكية لواجهة وتجربة المستخدم

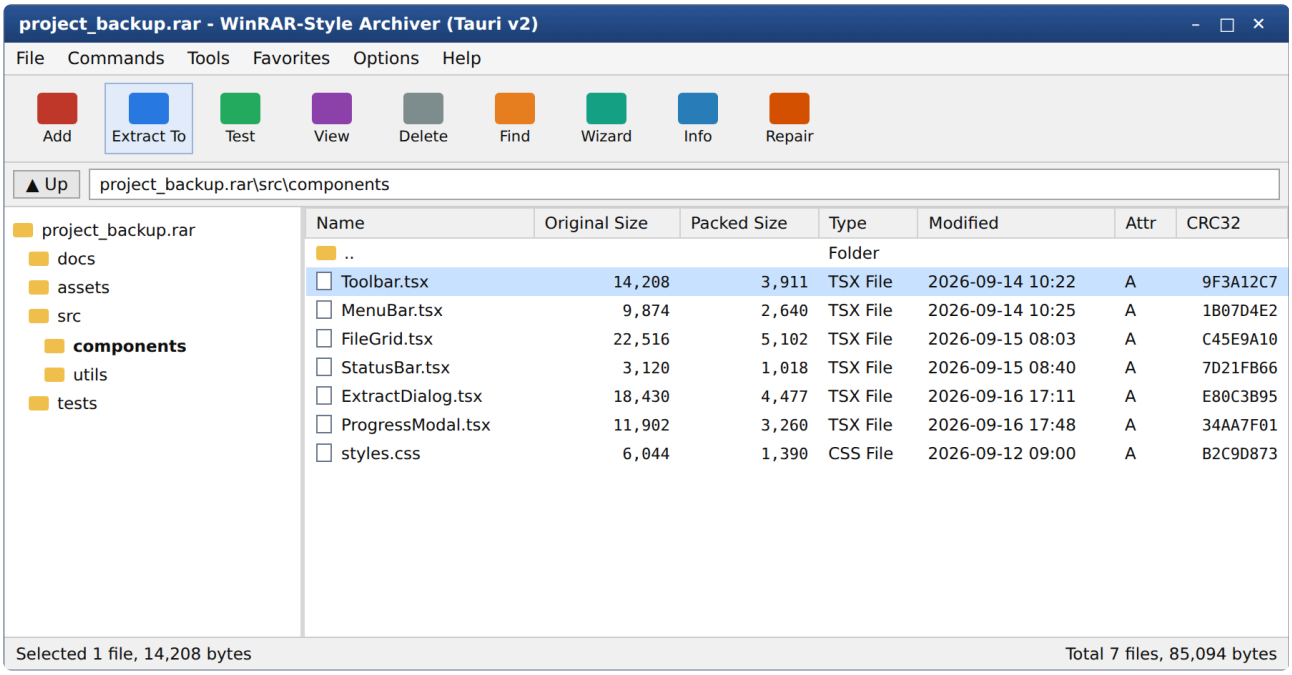
تلتزم برمجية WinRAR بفلسفة الواجهات المكتبية الكثيفة وعالية الإنتاجية، حيث يُفضل الوصول المباشر للبيانات والأدوات على الحركات البصرية المعقدة، مع الاحتفاظ بمرونة التخصيص الكامل للوحة التحكم وشبكة العرض.

2.1 الهيكل البصري وعناصر الواجهة الرئيسية

تقوم الشاشة الأساسية على ست طبقات بنيوية متسلسلة تبدأ بشريط القوائم القياسي الذي يضم تصنيفات الأوامر الرئيسية: ملف، أوامر، أدوات، المفضلة، خيارات، وتعليمات. يليه شريط الأدوات الرئيسي الذي يمتاز بقابلية التبديل بين الأزرار الكبيرة بأبعاد 48×36 بكسل والأزرار الصغيرة بأبعاد 24×24 بكسل، متضمناً أزرار الإجراءات الحيوية: إضافة، استخراج إلى، اختبار، عرض، حذف، بحث، المعالج، معلومات، وإصلاح.

يحتل شريط المسار التنقلي الموقع الفاصل بين شريط الأدوات وشبكة البيانات، متيحاً مساراً قابلاً للتحرير وزر الانتقال للمجلد الأب. ينقسم فضاء العمل الرئيسي رأسياً إلى جزءين عبر عارض تجزيئي مرن؛ يتولى الجانب الأيسر عرض الشجرة الهيكلية للمجلدات، بينما يشغل الجانب الأيمن جدول بيانات افتراضي يعرض الملفات مع أعمدة تفصيلية لاسم الملف، الحجم الأصلي، الحجم المضغوط، النوع، تاريخ التعديل، السمات، ورمز التدقيق الدوري.

يختتم البناء البصري بشريط الحالة السفلي، المسؤول عن توفير قراءات رقمية آنية لإجمالي عدد العناصر، وحجم البيانات الكلية، والبيانات المحددة حالياً.



الشكل (1): نموذج توضيحي للنافذة الرئيسية — شريط القوائم، شريط الأدوات، شريط المسار، الشجرة والجدول وشريط الحالة

المكون البصري	الاختصار الافتراضي	الوظيفة التقنية داخل منظومة العمل
إضافة (Add)	`Alt + A`	استدعاء نافذة تكوين الأرشيف وإعداد معاملات الضغط
استخراج إلى (Extract To)	`Alt + E`	فتح نافذة مسار وخيارات تفكيك البيانات
اختبار (Test)	`Alt + T`	فحص سلامة وتكامل كتل الأرشيف دون كتابة مادية على القرص
عرض (View)	`Alt + V`	إطلاق المعاين الداخلي للملفات أو تشغيل المعاين المقترن بالنظام
حذف (Delete)	`Del`	إزالة كائن أو مسار من داخل الأرشيف وإعادة بناء الفهرس
إصلاح (Repair)	`Alt + R`	تشغيل خوارزمية التعافي باستخدام سجلات ريد-سولومون
معلومات (Info)	`Alt + I`	استعراض التقرير الإحصائي والتشفيري ونسب الضغط للأرشيف
اختبار العتاد (Benchmark)	`Alt + B`	تقييم قدرة المعالج وقنوات الذاكرة على معالجة كتل الضغط

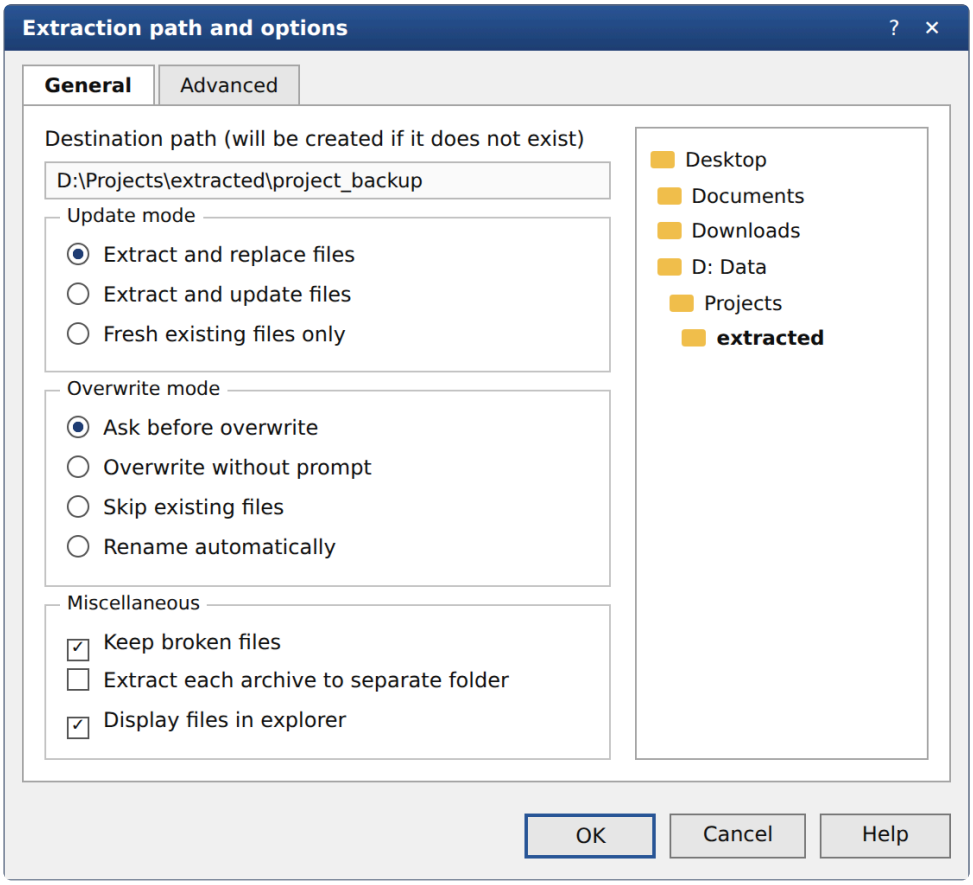
3 الحوارات التفاعلية المتقدمة ونماذج التشغيل

3.1 نافذة مسار وخيارات الاستخراج (Extraction Path and Options)

تعتبر هذه النافذة من أكثر النماذج تعقيداً في تجربة المستخدم؛ إذ تحتوي على مستعرض شجري لتحديد مسار الوجهة، مقترناً بلوحة تبويبات متعددة. يوفر التبويب العام خيارات التحكم في أنماط التحديث، والتي تتفرع إلى الاستخراج والاستبدال، أو الاستخراج والتحديث للملفات الأحدث، أو استخراج الملفات غير الموجودة فقط.

كما يدير أنماط الاستبدال التي تشمل السؤال قبل الاستبدال، أو الاستبدال الصامت المباشر، أو تخطي الملفات المتطابقة، أو إعادة التسمية التلقائية المتسلسلة. وتتضمن الخيارات العامة الإبقاء على الملفات التالفة جزئياً لاستنقاذ البيانات المقروءة بدلاً من مسحها، واستخراج كل أرشيف داخل مجلد فرعي مستقل، وفتح مستعرض النظام فور اكتمال المهمة.

أما التبويب المتقدم، فيتحكم في استعادة أذونات الأمان وقوائم الوصول، والاحتفاظ بالطوابع الزمنية الأصلية، ومسح ملف الأرشيف تلقائياً عقب انتهاء الاستخراج.

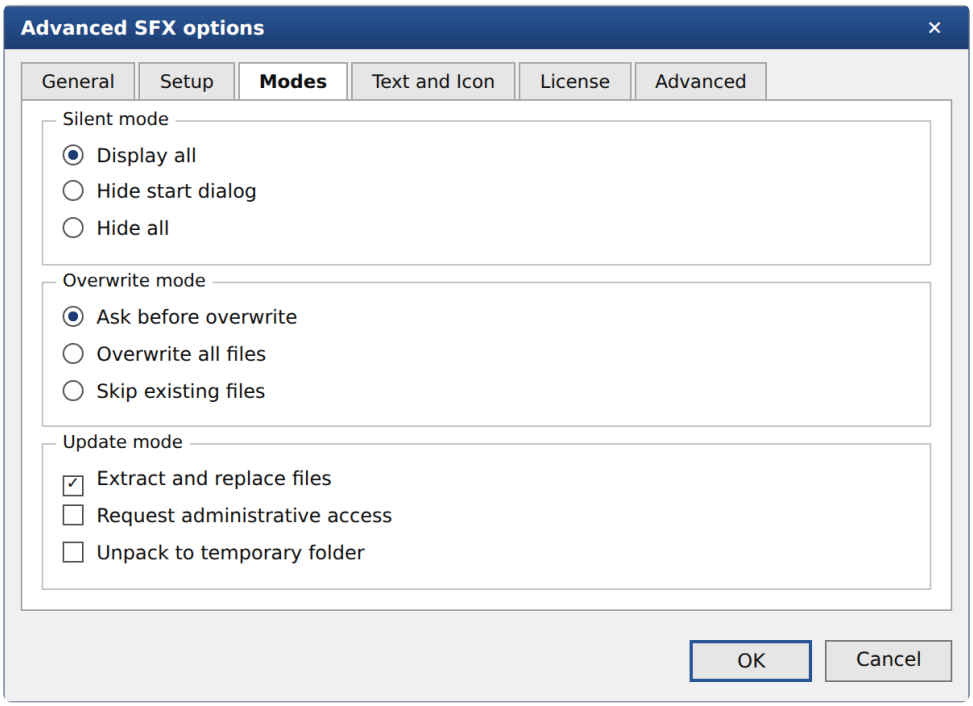


الشكل (2): نموذج توضيحي لنافذة مسار وخيارات الاستخراج — التبويب العام

3.2 إعدادات الأرشيف ذاتي الاستخراج المتقدم (Advanced SFX Options)

تتيح هذه المنظومة تحويل الأرشيف إلى ملف تنفيذي مستقل (`.exe`) يدمج شفرة فك ضغط أصلية (Stub) مع كتل البيانات، دون اشتراط وجود برمجية أرشفة في نظام العميل. يحتوي نموذج الإعداد على ستة تبويبات رئيسية:

- **عام:** لتحديد المسار الافتراضي وإمكانية فك الضغط في المسار المؤقت للنظام `%TEMP%`.
- **الإعداد:** لتسجيل ملف تنفيذي أو سكريبت يتم تشغيله آلياً فور اكتمال الاستخراج.
- **الأنماط:** للتحكم بدرجة ظهور العملية، ويتيح النمط الصامت بالكامل لإخفاء الواجهات كخدمة خلفية.
- **النصوص والأيقونات:** لإدراج شاشات الشرح، والشعارات الترويجية، واستبدال أيقونة الملف التنفيذي.
- **الترخيص:** لعرض شروط الاستخدام واشتراط موافقة المستخدم.
- **متقدم:** لجدولة حذف ملفات قديمة، وبناء اختصارات على سطح المكتب، وفرض طلب صلاحيات المسؤول الإدارية العالية (`-iadm`) لتفادي قيود التحكم في حساب المستخدم (UAC).

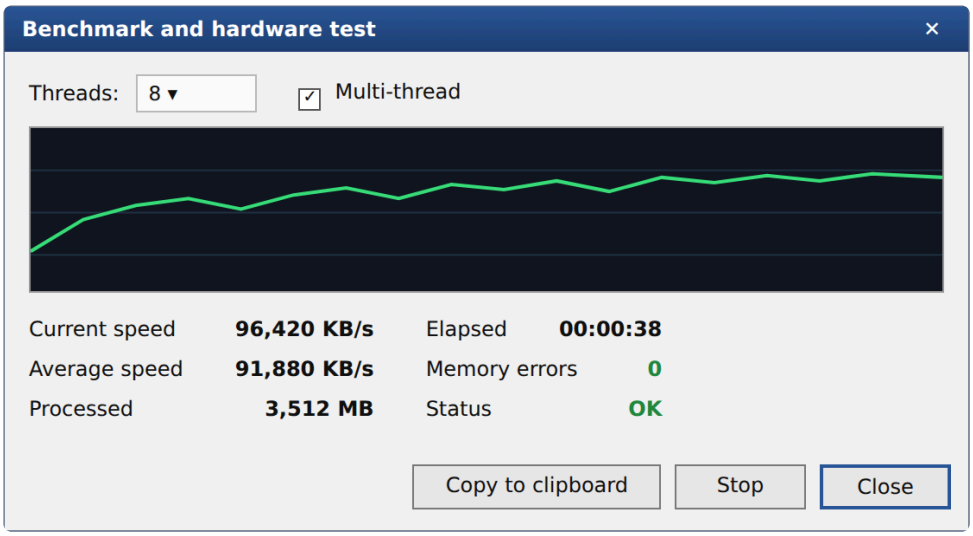


الشكل (3): نموذج توضيحي لنافذة إعدادات SFX المتقدمة — تبويب الأنماط

3.3 أداة قياس واختبار أداء العتاد (Benchmark and Hardware Test)

توفر هذه الأداة بيئة اختبار حوسبية متخصصة لقياس معدلات الضغط الحقيقية لعتاد الجهاز. تعرض النافذة السرعة اللحظية ومعدل السرعة الإجمالي بوحدة كيلوبايت/ثانية (KB/s)، مقترنة بحجم البيانات التي تم تمريرها والوقت المنقضي.

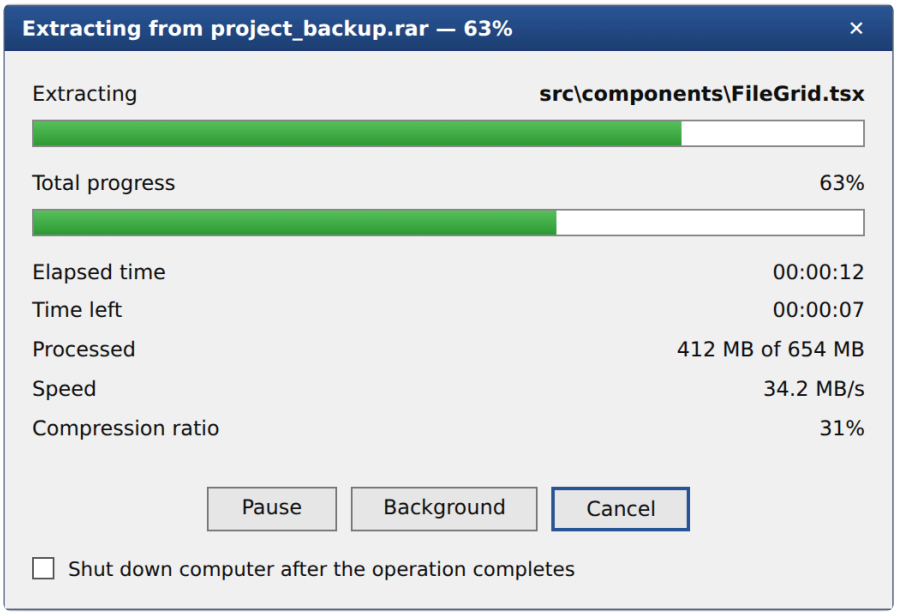
تدعم الأداة التبديل بين أنماط التشغيل أحادية الخيط ومتعددة الخيوط مع تحديد عدد الأنوية الموجهة للاختبار، وتتضمن شاشات مراقبة للأخطاء الحسابية للتحقق من سلامة قنوات نقل البيانات في الذاكرة تحت الضغط الشديد، مع زر لتصدير النتائج إلى الحافظة.



الشكل (4): نموذج توضيحي لأداة اختبار العتاد — السرعة اللحظية والمتوسطة وعداد أخطاء الذاكرة

3.4 نافذة تتبع العمليات غير المتزامنة (Operation Progress Modal)

تعتمد على شريطين منفصلين لقياس الإنجاز: شريط يرصد تقدم الملف المفرد الجاري معالجته، وشريط يتتبع التقدم التراكمي لمجموع الأرشيف. توفر النافذة بيانات مستمرة لمعدل تدفق البيانات المنقولة، والوقت المنقضي، والوقت المتبقي المقدر، مع أزرار للتحكم الحي تتيح الإيقاف المؤقت، وخفض أولوية المعالجة إلى نمط الخلفية منخفض الأعباء، أو الإغلاق التلقائي للنظام فور انتهاء المهمة.



الشكل (5): نموذج توضيحي لنافذة التقدم ذات الشريطين

4 المعمارية التقنية للمستنسخ الحديث

4.1 تفكيك معضلة الملكية الفكرية لصيغة RAR

تعتبر خوارزمية ضغط RAR وتنسيق كتابة وتوليد الأرشيف ملكية حصرية مغلقة لشركة win.rar GmbH والمهندس يوجين روشال. يحظر ترخيص كود المصدر الرسمي تفكيك أو إعادة هندسة خوارزمية الضغط لإنتاج أرشيفات RAR برمجياً بشكل مستقل. في المقابل، توفر الشركة الشفرة المصدرية لمكتبة فك الضغط (UNRAR Source Code) بترخيص مفتوح يسمح لأي مطور بقراءة واستخراج أرشيفات RAR دون قيود.

تتبع المشاريع مفتوحة المصدر الناجحة (مثل Zip-7 وPeaZip وZiplark وNanaZip) استراتيجية هندسية واضحة ومستقرة:

- **في عمليات القراءة والاستخراج:** يتم تضمين كود UNRAR الرسمي لفك تشفير واستخراج كافة أجيال RAR بما في ذلك الإصدارات الحديثة لـ RAR5 وRAR7.

- **في عمليات الإنشاء والضغط:** يتم توجيه المحرك لإنشاء تنسيقات الأرشفة المفتوحة وعالية الكفاءة، وبشكل أساسي معيار ZIP القياسي الممتد (Zip64)، وتنسيق 7z المعتمد على خوارزميات LZMA/LZMA2، مع إمكانية دعم تيار Zstandard للضغط الفائق، وتطبيق تشفير AES-256 المتوافق مع المعايير القياسية.

4.2 اختيار الحزمة البرمجية: Tauri v2 مقابل البدائل

تتطلب برمجيات الأرشفة معالجة مكثفة للبيانات وإدارة دقيقة للذاكرة المؤقتة، مما يجعل بيئات العمل القائمة على محرك Chromium مثل Electron خياراً غير ملائم؛ نظراً لاستهلاكها المرتفع للذاكرة، وحجم الحزمة الكبير، واختناقات خيوط المعالجة الناتجة عن محرك V8. ورغم أن بيئة .NET مع مكتبة SharpCompress تقدم حلاً متكاملاً لإدارة تدفقات الضغط المتعددة، فإنها تظل مقيدة نسبياً ببيئة ويندوز.

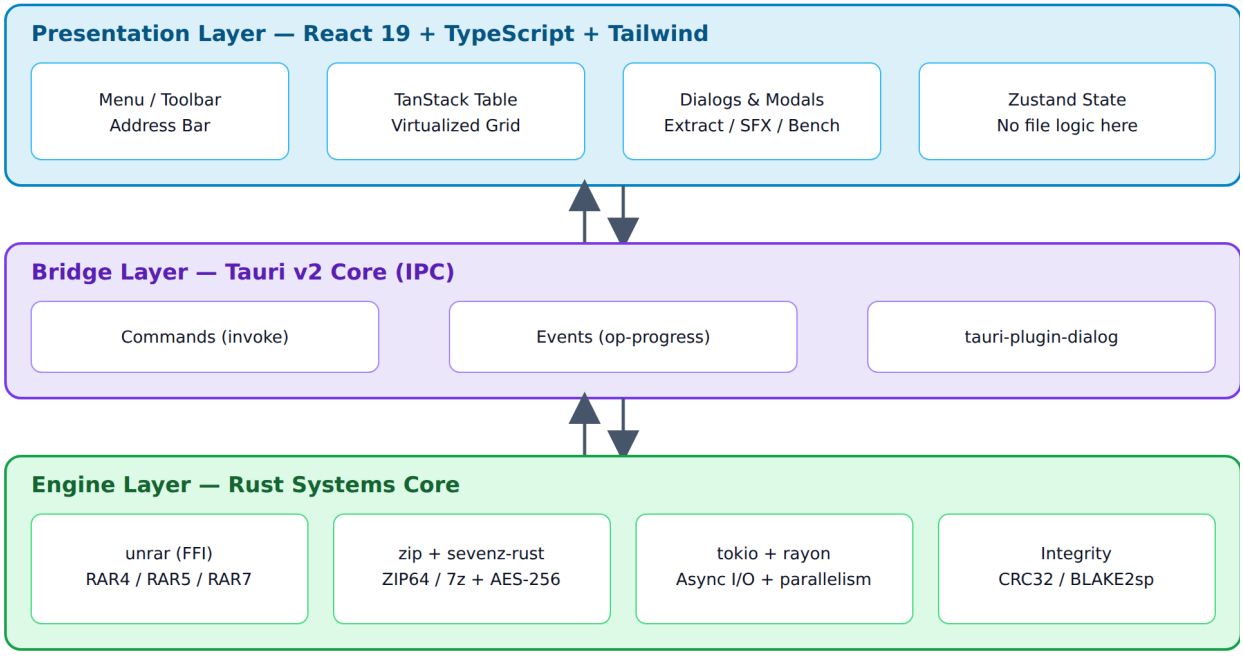
يمثل إطار عمل Tauri v2 مقترناً بلغة Rust في الواجهة الخلفية وTypeScript / React 19 في الواجهة الأمامية النموذج الأنسب للاستنساخ. يوفر هذا التكامل حجماً تنفيذياً لا يتجاوز 10 ميجابايت، واستهلاكاً للذاكرة يقل عن 20 ميجابايت، مع الاستفادة من محركات العرض الأصلية لنظام التشغيل. كما تتيح لغة Rust أداءً قريباً من العتاد يضاهي لغة C++ المستخدمة في بناء WinRAR الأصلي، مع ضمان سلامة الذاكرة وتسهيل المعالجة المتوازية عبر الخيوط المتعددة.

الطبقة المعمارية	التقنية المختارة	الدور الفني المباشر	مبرر الاختيار الهندسي
محرك فك الضغط	`unrar crate (C FFI)`	قراءة واستخراج أرشيفات RAR4 وRAR5 وRAR7	الاعتماد المباشر على كود المصدر الرسمي غير المقيد لفك الضغط
محرك الضغط المفتوح	`zip, sevenz-rust`	إنشاء أرشيفات ZIP وZ7 مع تشفير AES-256	مرونة التنسيقات المفتوحة وتفادي التعقيدات القانونية لضغط RAR
جدولة المهام والتدفق	`tokio, rayon`	معالجة عمليات الإدخال/الإخراج واختبار الأداء	التوازي الحقيقي وتفادي تجميد الواجهة أثناء المهام الكثيفة
جسر التواصل (IPC)	`Tauri v2 Core Events`	بث مؤشرات التقدم عبر قنوات غير متزامنة	كفاءة نقل البيانات بين محرك Rust وواجهة العرض
شبكة عرض الملفات	`@tanstack/react-table`	تصيير مصفوفات الملفات الشجرية بأسلوب افتراضي	معالجة آلاف العناصر بسلاسة دون هبوط معدل الإطارات
إدارة الحالة والواجهة	`zustand + tailwindcss`	ضبط إعدادات السمات، والمسارات، وحوارات التفاعل	فصل الحالة البرمجية وسهولة محاكاة المظهر الكلاسيكي

4.3 المعمارية الطبقية للمنظومة

تنتظم المنظومة في ثلاث طبقات متماسكة تتصل عبر قنوات تبادل بيانات صلبة:

- **طبقة العرض والواجهة (Presentation Layer):** تُبنى باستخدام React 19 ونظام تصميم Tailwind CSS. تتولى محاكاة المظهر الكلاسيكي المحسن لبرمجية WinRAR من أشرطة أدوات، وحوارات مبوبة، وجداول بيانات افتراضية، دون احتواء أي منطق لفك شفرات الملفات أو معالجة البايتات.
- **طبقة الجسر والتحكم (Tauri Application Layer):** تدير دورة حياة النوافذ، وتستدعي نوافذ الحوار الأصلية الخاصة بنظام التشغيل عبر مكتبة `tauri-plugin-dialog` لفتح المجلدات وحفظ الملفات، وتعمل كمنظم للرسائل المنقولة بين العمليات (IPC).
- **طبقة المحرك والنواة (Rust Systems Engine):** تتولى العمليات الحوسبية الحقيقية. تقرأ الترويسات الثنائية للأرشيفات، وتفك تشفير كتل البيانات، وتتحقق من صحة مفاتيح التشفير وتجزئة BLAKE2sp، وتبث أحداث التقدم اللحظية عبر قنوات Tokio غير المتزامنة دون حظر خيط التنفيذ الأساسي.



الشكل (6): مخطط المعمارية الطبقية — العرض، الجسر، المحرك

5 إطار عمل واستراتيجية البرمجة الموجهة (Vibe Coding)

تعتمد البرمجة الموجهة في المشاريع المعقدة على صياغة سياق فني محكم يوجه أدوات الذكاء الاصطناعي (مثل Cursor وWindsurf) للالتزام بالأنماط المعمارية الصحيحة، وتفادي كتابة أكواد غير متوافقة أو الاعتماد على مكتبات غير ملائمة للمهام منخفضة المستوى.

5.1 وثيقة مواصفات المنتج والسياق الفني (PRD.md)

يتم وضع هذه الوثيقة داخل الملف المرجعي الأساسي في جذر المشروع لتكون بمثابة دستور للتطوير يلتزم به الذكاء الاصطناعي طوال مراحل التوليد:

```
Product Requirements Document (PRD) - System Archiver Desktop Application

1. Scope & Objective
High-performance cross-platform archive manager mimicking WinRAR's functional
workflows and UI density, engineered using Tauri v2, Rust, React 19, and Tailwind CSS.

2. Technical Stack Boundaries
Backend Engine: Rust (stable edition), Tokio async runtime, unrar-sys FFI,
                zip-rs, sevenz-rust, rayon.
Presentation:   React 19, TypeScript, Tailwind CSS, TanStack Virtual/Table, Zustand.
Architecture:   Zero file processing on the UI thread. All disk read/writes must
                stream through Rust channels with progress emission.

3. Supported Formats Policy
Decompression / Inspection: Full support for RAR (RAR4, RAR5, RAR7), ZIP, TAR, GZ, and 7z.
Creation / Compression: ZIP (with Zip64 support) and 7z with AES-256 encryption.
Creation of proprietary RAR is out of scope due to license restrictions.

4. UI/UX Specifications
Retro-modern aesthetic: Dual-density toolbars (large 48x36px and small 24x24px),
tabular data view, split-pane folder hierarchy.
Modals required: Extraction Options (with overwrite/update modes), SFX Configuration,
Benchmark Tool, and Dual-Progress Bar monitor.
```

5.2 الأوامر البرمجية التنفيذية للبناء التدريجي (Master Prompts Catalog)

يتحقق نجاح الاستنساخ عبر تقسيم عملية التوجيه إلى أربعة موجهات ذرية، تُطرح بالتتابع بعد استقرار كل مرحلة تقنية والتحقق من سلامة البناء واختبارات التشغيل.

الموجه الأول: تأسيس بيئة العمل وهيكلة الواجهة البصرية

يوجه هذا الأمر لبناء البنية الأمامية المتكاملة، وتثبيت عناصر الواجهة الكلاسيكية قبل إقحام أي منطق برمجي في النواة:

```
Act as a Principal Desktop Frontend Engineer. Initialize the client interface for our
desktop file archiver inside the Tauri v2 repository, using React 19, TypeScript, and
Tailwind CSS.

Create a modular, high-density desktop layout replicating the structure of WinRAR:

- Classic Menu Bar at the very top: Implement dropdowns for "File", "Commands", "Tools",
  "Favorites", "Options", and "Help". Include standard keyboard shortcut cues
  (e.g., Alt+A for Add, Alt+E for Extract, Alt+T for Test).
- Main Action Toolbar: Create a toolbar supporting a display toggle between large buttons
  (48x36px) and small buttons (24x24px). Populate it with high-contrast SVG action icons:
  "Add", "Extract To", "Test", "View", "Delete", "Find", "Wizard", "Info", and "Repair".
- Address and Navigation Bar: Include a clickable path breakdown, an editable path input,
  and an "Up One Level" navigation button.
- Resizable Split Workspace:
    Left Pane:  A collapsible Folder Tree view for directory hierarchy traversal.
    Right Pane: A virtualized data grid (TanStack Table) displaying file rows with
columns:
                "Name", "Original Size", "Packed Size", "Type", "Modified Date",
                "Attributes", and "CRC32". Monospace alignment for sizes and dates.
- Bottom Status Bar: Dynamic display showing total file count, total uncompressed size,
  and current selection summary.

Style everything using pure Tailwind CSS with a clean Windows desktop theme (light neutral
slate, subtle 1px border dividers, dense padding). Use mock data for testing and ensure
zero external UI kit dependencies.
```

الموجه الثاني: بناء محرك فحص وقراءة الأرشيف وربط قنوات الاتصال

يوجه لربط واجهة العرض بالنواة الخلفية في Rust لفتح الأرشيفات، واستخراج الترويسات، وإتاحة التصفح الشجري للمحتويات دون كتابة الملفات على القرص:

```
Act as a Systems Programmer specializing in Rust and Tauri v2. Implement the archive
inspection and navigation subsystem.

Requirements:
1. In src-tauri/Cargo.toml, add: unrar, zip, sevenz-rust, serde, serde_json, chrono,
tokio.
2. Define a serializable Rust struct ArchiveEntry:
     name: String, path: String, is_dir: bool, original_size: u64,
     packed_size: u64, modified: String, crc: String, is_encrypted: bool
3. Implement a Tauri command
     #[tauri::command] async fn open_archive(path: String) -> Result, String>
   - Identify format via magic bytes (.rar, .zip, .7z).
   - For RAR files: invoke the unrar crate to inspect headers of RAR4, RAR5, and RAR7
     without extracting payload bytes to disk.
```

```
   - For ZIP and 7z: inspect central directories and extract entry metadata.
   - Gracefully catch and return specific errors if archive headers are encrypted
     (prompting password requirements).
4. Integrate with the React frontend:
   - Connect native OS drag-and-drop to pass dropped archive paths to open_archive.
   - Implement an in-archive directory traversal state in Zustand: double-clicking a
     folder updates the view to display that subfolder's entries, updates the Address Bar,
     and enables the "Up One Level" button.
```

الموجه الثالث: محرك الاستخراج والضغط وتدفق مؤشرات الإنجاز

يوجه لبناء نوافذ الحوار التفصيلية وعمليات المعالجة غير المتزامنة الكبرى، مع ضمان بث تدفق البيانات بنمط غير حاظر للواجهة:

```
Act as a Full-Stack Systems Engineer. Implement the full asynchronous Extraction and
Compression workflows with real-time progress tracking.

Requirements:
1. React Extraction Dialog:
   - Modal matching WinRAR's "Extraction Path and Options".
   - Include a destination folder picker using tauri-plugin-dialog.
   - "General" tab options: Update modes (Extract and replace, Extract and update,
Freshen),
     Overwrite modes (Ask before overwrite, Overwrite without prompt, Skip existing),
     checkboxes for "Keep broken files" and "Open folder after extraction".
2. Operation Progress Modal:
   - A dual-progress bar window showing: "Current File Progress" and "Total Progress".
   - Data indicators: current file path, processed volume vs total volume,
     processing rate (MB/s), and elapsed/estimated time.
   - Operational buttons: "Pause", "Background", and "Cancel".
3. Rust Execution Engine:
   - Implement extract_archive(config: ExtractionConfig) executing on a Tokio
     background thread.
   - Implement create_archive(config: CompressionConfig) capable of writing ZIP64 or 7z
     with optional AES-256 encryption.
   - Emit a Tauri event "op-progress" on every buffer chunk processed to stream byte
     progress to the frontend without UI lag.
   - Obey the keep_broken_files flag: preserve corrupted target files if a checksum fails
     instead of automatically removing them.
```

الموجه الرابع: وحدات الفحص التخصصية وتوليد الأرشيف ذاتي الاستخراج

يوجه لبناء أدوات التشخيص المتقدمة ومولد الملفات التنفيذية المستقلة:

```
Act as an Advanced Systems Architect. Implement the Hardware Benchmark Tool and the
Self-Extracting Archive (SFX) creation subsystem.

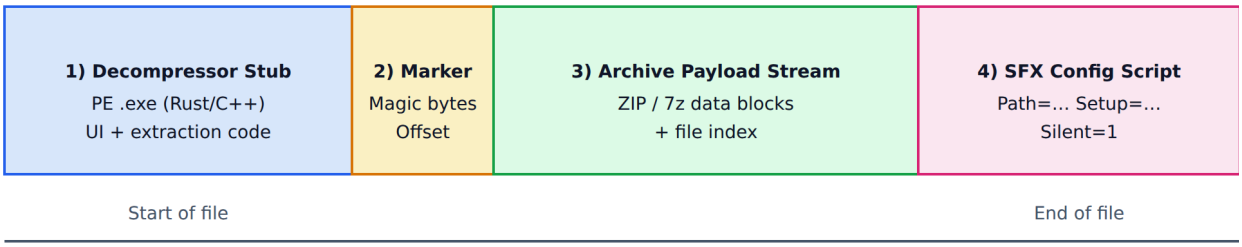
Requirements:
1. Benchmark and Hardware Test Module:
   - Rust Engine: Build a throughput benchmark using rayon. Allocate a memory buffer,
     execute parallel synthetic compression and hashing passes, and calculate processing
```

```
      speed in KB/s.
    - Expose commands start_benchmark(threads: usize) and stop_benchmark().
      Include dynamic memory integrity validation counters.
    - Frontend: Build a modal matching WinRAR's Benchmark dialog displaying real-time
      speed, average speed, processed volume, thread count selector, memory error
      indicator, and a "Copy to Clipboard" button.
2. Advanced SFX Builder:
    - Frontend: Build the "Advanced SFX Options" modal with tabs: "General" (Extraction
      path), "Setup" (Executable to run after unpacking), "Modes" (Silent options:
      Display all, Hide start, Hide all), "Text and Icon" (Custom title and message),
      and "Advanced" (Request administrative privileges).
    - Rust Generator: Create a stub concatenation engine that combines a pre-compiled
      lightweight decompressor executable stub with a compressed ZIP/7z payload and an
      SFX metadata script appended to the binary overlay.
```

6 الهندسة الداخلية لملفات الأرشيف ذاتي الاستخراج (SFX)

يقوم البناء المعماري للأرشيف ذاتي الاستخراج على دمج كتل ثنائية متسلسلة داخل ملف تنفيذي قياسي (Portable Executable - PE):

1. **كتلة الملف التنفيذي المسبق البناء (Decompressor Stub):** تمثل رأس الملف التنفيذي الأساسي، وتحتوي على شفرة تشغيلية مصمتة بلغة Rust أو C++ تتولى واجهة العرض المصغرة واستدعاء دوال فك الضغط الأساسية.
2. **العلامة الإشارية (Payload Signature Marker):** فاصل ثنائي محدد (مثل بايتات سحرية مخصصة) يتيح للبرنامج التنفيذي تحديد الإزاحة المكانية (Offset) التي ينتهي عندها كود التشغيل وتبدأ عندها بيانات الأرشيف.
3. **حمولة الأرشيف المضغوطة (Archive Payload Stream):** تيار البيانات الخام للأرشيف المضغوط (تنسيق ZIP أو 7z) متضمناً كتل الملفات وتفاصيل الفهرسة.
4. **شفرة التهيئة الوصفية (SFX Configuration Script):** كتلة نصية توضع في نهاية الملف لتحديد مسار الاستخراج الافتراضي، وأوامر التشغيل اللاحقة (مثل `Setup=installer.exe`)، ودرجة الإخفاء الصامت للواجهة.



الشكل (7): البنية الثنائية لملف SFX — من بداية الملف إلى نهايته

عند قيام المستخدم النهائي بتشغيل ملف SFX، يشرع كود الـ Stub بالعمل أولاً؛ حيث يقرأ البرنامج ملفه التنفيذي الخاص عبر قنوات القراءة الثنائية من القرص (`std::env::current_exe()`)، ثم يقفز مباشرة إلى الإزاحة المحددة بالعلامة الإشارية، ويستخرج كتل البيانات إلى المسار المحدد بالتهيئة، ويطلق العمليات البرمجية اللاحقة تلقائياً قبل إنهاء مهمته.

7 الاستنتاجات الهندسية والتوصيات التنفيذية

يكشف التحليل الفني لمنظومة WinRAR أن تميزها التاريخي لم يكن وليد صدفة، بل نتاج توازن معماري دقيق يجمع بين خوارزميات أرشفة قوية، وتكامل منخفض المستوى مع أنظمة التشغيل، وواجهات مستخدم تضع الكفاءة فوق الزخارف الشكلية. يتطلب استنساخ هذه المنظومة بنجاح عبر أسلوب البرمجة الموجهة مراعاة المعايير الآتية:

الفصل الوظيفي والقانوني في معالجة التنسيقات: يجب الالتزام بفك ضغط كافة أنواع الملفات (بما فيها RAR بكافة أجياله) باستخدام كود مكتبة UNRAR المفتوح، مع حصر وظائف الضغط والإنشاء في المعايير الحرة مثل ZIP64 و7z لتفادي انتهاك الملكية الفكرية وتجاوز القيود القانونية لصيغة RAR المغلقة.

حتمية الأداء في المعمارية الأساسية: تفرض متطلبات الذاكرة وقواميس الضغط الكبيرة الاعتماد على بيئة تقنية خفيفة وسريعة مثل Tauri v2 ولغة Rust؛ حيث تمنح هذه التوليفة قدرات تحكم كاملة في المعالجة متعددة الخيوط وإدارة الذاكرة، مع الحفاظ على زمن استجابة لا يتجاوز 16 ميلي ثانية لتصيير واجهة المستخدم.

الانضباط الهندسي في سياقات التوليد الآلي: تتطلب البرمجة الموجهة تجنب الموجهات العريضة والشاملة، والاعتماد بدلاً من ذلك على وثيقة مواصفات المنتج (PRD) وتمرير الأوامر الذرية المتدرجة التي تفصل بناء الواجهات عن منطق النواة، مما يضمن الحصول على برمجية مستقرة وعالية الأداء تضاهي حلول الأرشفة الاحترافية.